

Chapter 14 Appendix

14.1 Preface

14.1.1 Formatting

The following `ggplot2` theme was use to format plots in this book:

```
plot_style <- function() {  
  font <- "Helvetica"  
  
  ggplot2::theme_classic() +  
  ggplot2::theme(  
    plot.title = ggplot2::element_text(  
      family = font, size=14, color = "#222222"),  
    plot.subtitle = ggplot2::element_text(  
      family=font, size=12, color = "#666666"),  
  
    legend.position = "right",  
    legend.background = ggplot2::element_blank(),  
    legend.title = ggplot2::element_blank(),  
    legend.key = ggplot2::element_blank(),  
    legend.text = ggplot2::element_text(  
      family=font, size=14, color="#222222"),  
  
    axis.title.y = ggplot2::element_text(  
      margin = ggplot2::margin(t = 0, r = 8, b = 0, l = 0),  
      size = 14, color="#666666"),  
    axis.title.x = ggplot2::element_text(  
      margin = ggplot2::margin(t = -2, r = 0, b = 0, l = 0),  
      size = 14, color = "#666666"),  
    axis.text = ggplot2::element_text(  
      family=font, size=14, color="#222222"),  
    axis.text.x = ggplot2::element_text(  
      margin = ggplot2::margin(5, b = 10)),  
    axis.ticks = ggplot2::element_blank(),  
    axis.line = ggplot2::element_blank(),  
  
    panel.grid.minor = ggplot2::element_blank(),  
    panel.grid.major.y = ggplot2::element_line(color = "#eeeeee"),  
    panel.grid.major.x = ggplot2::element_line(color = "#ebebeb"),  
  
    panel.background = ggplot2::element_blank(),  
  
    strip.background = ggplot2::element_rect(fill = "white"),  
    strip.text = ggplot2::element_text(size = 20, hjust = 0)
```

```
)
}
```

Which you can then active with:

```
ggplot2::theme_set(plot_style())
```

14.2 Introduction

14.2.1 Worlds Store Capacity

The following script was used to generate the worlds store capacity diagram:

```
library(ggplot2)
library(dplyr)
library(tidyr)
read.csv("data/01-worlds-capacity-to-store-information.csv", skip = 8) %>%
  gather(key = storage, value = capacity, analog, digital) %>%
  mutate(year = X, terabytes = capacity / 1e+12) %>%
  ggplot(aes(x = year, y = terabytes, group = storage)) +
    geom_line(aes(linetype = storage)) +
    geom_point(aes(shape = storage)) +
    scale_y_log10(
      breaks = scales::trans_breaks("log10", function(x) 10^x),
      labels = scales::trans_format("log10", scales::math_format(10^x))
    ) +
    theme_light() +
    theme(legend.position = "bottom")
```

14.2.2 Daily downloads of CRAN packages

The CRAN downloads chart was generated through:

```

downloads_csv <- "data/01-intro-r-cran-downloads.csv"
if (!file.exists(downloads_csv)) {
  downloads <- cranlogs::cran_downloads(from = "2014-01-01", to = "2019-01-01")
  readr::write_csv(downloads, downloads_csv)
}

cran_downloads <- readr::read_csv(downloads_csv)

ggplot(cran_downloads, aes(date, count)) +
  labs(title = "CRAN Packages",
       subtitle = "Total daily downloads over time") +
  geom_point(colour="black", pch = 21, size = 1) +
  scale_x_date() + xlab("year") + ylab("downloads") +
  scale_x_date(date_breaks = "1 year",
              labels = function(x) substring(x, 1, 4)) +
  scale_y_continuous(
    limits = c(0, 3.5 * 10^6),
    breaks = c(0.5 * 10^6, 10^6, 1.5 * 10^6, 2 * 10^6, 2.5 * 10^6, 3 * 10^6, 3.5 * 10^6),
    labels = c("", "1M", "", "2M", "", "3M", "")
  )

```

14.3 Getting Started

14.3.1 Prerequisites

14.3.1.1 Installing R

From r-project.org, download and launch the R installer for your platform, Windows, Macs or Linux available.

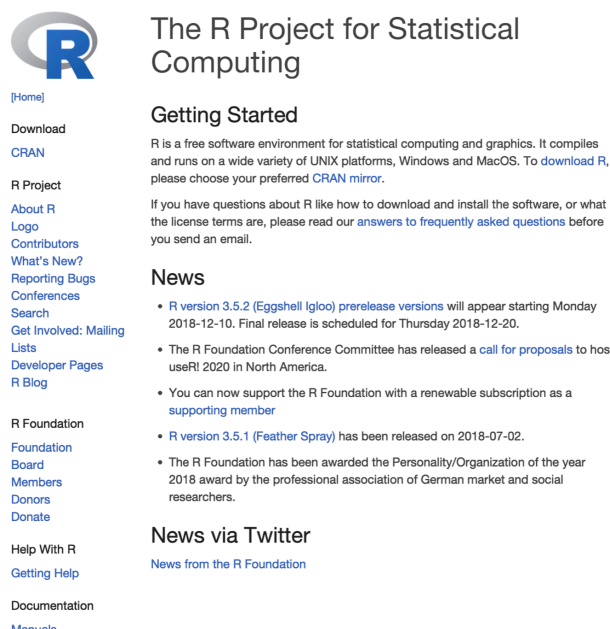


FIGURE 14.1: The R Project for Statistical Computing

14.3.1.2 Installing Java

From java.com/download, download and launch the installer for your platform, Windows, Macs or Linux are also available.

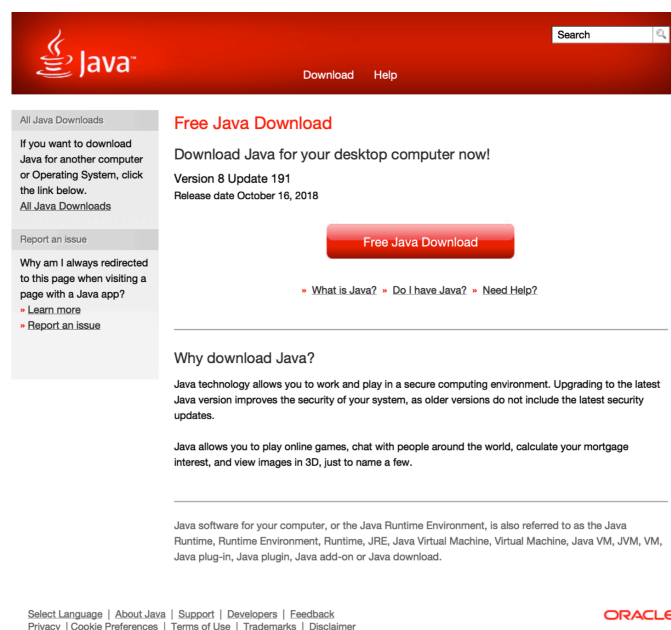


FIGURE 14.2: Java Download Page

Starting with Spark 2.1, Java 8 is required; however, previous versions of Spark support Java 7. Regardless, we recommend installing Java Runtime Engine 8, or JRE 8 for short.

Note: For advanced readers that are already using the Java Development Kit, JDK for short. Please notice that JDK 9+ is currently unsupported so you will need to downgrade to JDK 8 by uninstalling JDK 9+ or by setting `JAVA_HOME` appropriately.

14.3.1.3 Installing RStudio

While installing RStudio is not strictly required to work with Spark with R, it will make you much more productive and therefore, I would recommend you take the time to install RStudio from rstudio.com/download, then download and launch the installer for your platform: Windows, Macs or Linux.

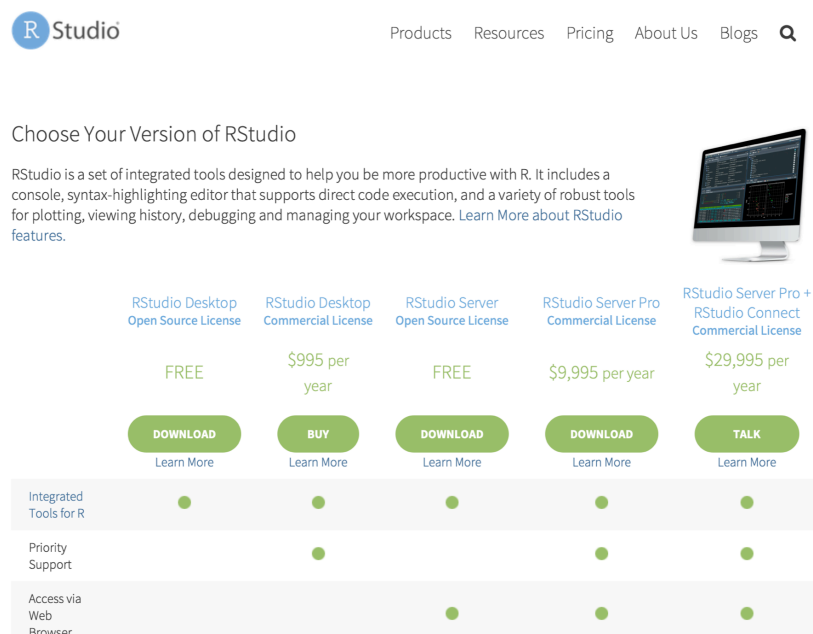


FIGURE 14.3: RStudio Downloads Page

After launching RStudio, you can use RStudio's console panel to execute the code provided in this chapter.

14.3.1.4 Using RStudio

If you are not familiar with RStudio, you should make note of the following panes:

- **Console:** A standalone R console you can use to execute all the code presented in this book.
- **Packages:** This pane allows you to install `sparklyr` with ease, check its version, navigate to the help contents, etc.
- **Connections:** This pane allows you to connect to Spark, manage your active connection and view the available datasets.

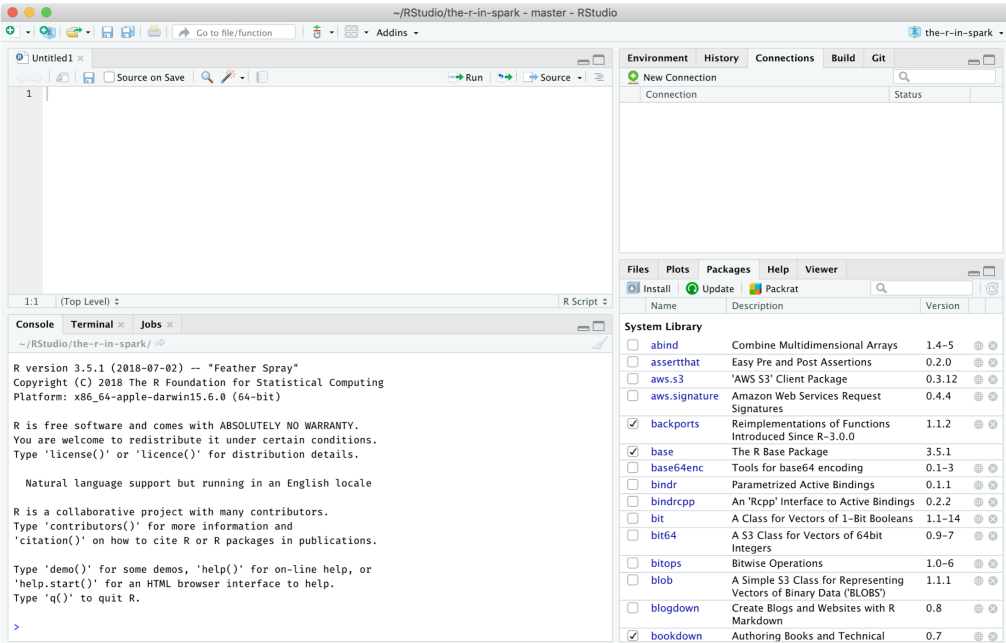


FIGURE 14.4: RStudio Overview

14.4 Analysis

14.4.1 Hive Functions

Name	Description
<code>size(Map<K.V>)</code>	Returns the number of elements in the map type.
<code>size(Array)</code>	Returns the number of elements in the array type.
<code>map_keys(Map<K.V>)</code>	Returns an unordered array containing the keys of the input map.
<code>map_values(Map<K.V>)</code>	Returns an unordered array containing the values of the input map.
<code>array_contains(Array, value)</code>	Returns TRUE if the array contains value.
<code>sort_array(Array)</code>	Sorts the input array in ascending order according to the natural ordering of the array elements and returns it
<code>binary(string or binary)</code>	Casts the parameter into a binary.
<code>cast(expr as 'type')</code>	Converts the results of the expression <code>expr</code> to the given type.
<code>from_unixtime(bigint unixtime[, string format])</code>	Converts the number of seconds from unix epoch (1970-01-01 00:00:00 UTC) to a string.
<code>unix_timestamp()</code>	Gets current Unix timestamp in seconds.
<code>unix_timestamp(string date)</code>	Converts time string in format yyyy-MM-dd HH:mm:ss to Unix timestamp (in seconds).
<code>to_date(string timestamp)</code>	Returns the date part of a timestamp string.
<code>year(string date)</code>	Returns the year part of a date.
<code>quarter(date/timestamp/string)</code>	Returns the quarter of the year for a date.

Name	Description
month(string date)	Returns the month part of a date or a timestamp string.
day(string date) dayofmonth(date)	Returns the day part of a date or a timestamp string.
hour(string date)	Returns the hour of the timestamp.
minute(string date)	Returns the minute of the timestamp.
second(string date)	Returns the second of the timestamp.
weekofyear(string date)	Returns the week number of a timestamp string.
extract(field FROM source)	Retrieve fields such as days or hours from source. Source must be a date, timestamp, interval or a string that can be converted into either a date or timestamp.
datediff(string enddate, string startdate)	Returns the number of days from startdate to enddate.
date_add(date/timestamp/string startdate, tinyint/smallint/int days)	Adds a number of days to startdate.
date_sub(date/timestamp/string startdate, tinyint/smallint/int days)	Subtracts a number of days to startdate.
from_utc_timestamp({any primitive type} ts, string timezone)	Converts a timestamp in UTC to a given timezone.
to_utc_timestamp({any primitive type} ts, string timezone)	Converts a timestamp in a given timezone to UTC.
current_date	Returns the current date.
current_timestamp	Returns the current timestamp.
add_months(string start_date, int num_months, output_date_format)	Returns the date that is num_months after start_date.
last_day(string date)	Returns the last day of the month which the date belongs to.

Name	Description
<code>next_day(string start_date, string day_of_week)</code>	Returns the first date which is later than <code>start_date</code> and named as <code>day_of_week</code> .
<code>trunc(string date, string format)</code>	Returns date truncated to the unit specified by the format.
<code>months_between(date1, date2)</code>	Returns number of months between dates <code>date1</code> and <code>date2</code> .
<code>date_format(date/timestamp/string ts, string fmt)</code>	Converts a date/timestamp/string to a value of string in the format specified by the date format <code>fmt</code> .
<code>if(boolean testCondition, T valueTrue, T valueFalseOrNull)</code>	Returns <code>valueTrue</code> when <code>testCondition</code> is true, returns <code>valueFalseOrNull</code> otherwise.
<code>isnull(a)</code>	Returns true if <code>a</code> is NULL and false otherwise.
<code>isnotnull (a)</code>	Returns true if <code>a</code> is not NULL and false otherwise.
<code>nvl(T value, T default_value)</code>	Returns default value if value is null else returns value.
<code>COALESCE(T v1, T v2, ...)</code>	Returns the first <code>v</code> that is not NULL, or NULL if all <code>v</code> 's are NULL.
<code>CASE a WHEN b THEN c [WHEN d THEN e]* [ELSE f] END</code>	When <code>a = b</code> , returns <code>c</code> ; when <code>a = d</code> , returns <code>e</code> ; else returns <code>f</code> .
<code>nullif(a, b)</code>	Returns NULL if <code>a=b</code> ; otherwise returns <code>a</code> .
<code>assert_true(boolean condition)</code>	Throw an exception if 'condition' is not true, otherwise return null.
<code>ascii(string str)</code>	Returns the numeric value of the first character of <code>str</code> .
<code>base64(binary bin)</code>	Converts the argument from binary to a base 64 string.
<code>character_length(string str)</code>	Returns the number of UTF-8 characters contained in <code>str</code> .

Name	Description
<code>chr(bigint</code>	<code>double A)</code>
<code>concat(string</code>	<code>binary A, string</code>
<code>context_ngrams(array<array>, array, int K, int pf)</code>	Returns the top-k contextual N-grams from a set of tokenized sentences.
<code>concat_ws(string SEP, string A, string B...)</code>	Like <code>concat()</code> above, but with custom separator SEP.
<code>decode(binary bin, string charset)</code>	Decodes the first argument into a String using the provided character set (one of 'US-ASCII', 'ISO-8859-1', 'UTF-8', 'UTF-16BE', 'UTF-16LE', 'UTF-16'). If either argument is null, the result will also be null.
<code>elt(N int, str1 string, str2 string, str3 string, ...)</code>	Return string at index number, <code>elt(2, 'hello', 'world')</code> returns 'world'.
<code>encode(string src, string charset)</code>	Encodes the first argument into a BINARY using the provided character set (one of 'US-ASCII', 'ISO-8859-1', 'UTF-8', 'UTF-16BE', 'UTF-16LE', 'UTF-16').
<code>field(val T, val1 T, val2 T, val3 T, ...)</code>	Returns the index of val in the <code>val1, val2, val3, ...</code> list or 0 if not found.
<code>find_in_set(string str, string strList)</code>	Returns the first occurrence of str in strList where strList is a comma-delimited string.
<code>format_number(number x, int d)</code>	Formats the number X to a format like '#,###,###.##' , rounded to D decimal places, and returns the result as a string. If D is 0, the result has no decimal point or fractional part.
<code>get_json_object(string json_string, string path)</code>	Extracts json object from a json string based on json path specified, and returns json string of the extracted json object.
<code>in_file(string str, string filename)</code>	Returns true if the string str appears as an entire line in filename.

Name	Description
<code>instr(string str, string substr)</code>	Returns the position of the first occurrence of substr in str.
<code>length(string A)</code>	Returns the length of the string.
<code>locate(string substr, string str[, int pos])</code>	Returns the position of the first occurrence of substr in str after position pos.
<code>lower(string A) lcase(string A)</code>	Returns the string resulting from converting all characters of B to lower case.
<code>lpad(string str, int len, string pad)</code>	Returns str, left-padded with pad to a length of len. If str is longer than len, the return value is shortened to len characters.
<code>ltrim(string A)</code>	Returns the string resulting from trimming spaces from the beginning(left hand side) of A.
<code>ngrams(array<array>, int N, int K, int pf)</code>	Returns the top-k N-grams from a set of tokenized sentences, such as those returned by the sentences() UDAF.
<code>octet_length(string str)</code>	Returns the number of octets required to hold the string str in UTF-8 encoding.
<code>parse_url(string urlString, string partToExtract [, string keyToExtract])</code>	Returns the specified part from the URL. Valid values for partToExtract include HOST, PATH, QUERY, REF, PROTOCOL, AUTHORITY, FILE, and USERINFO.
<code>printf(String format, Obj... args)</code>	Returns the input formatted according do printf-style format strings.
<code>regexp_extract(string subject, string pattern, int index)</code>	Returns the string extracted using the pattern.
<code>regexp_replace(string INITIAL_STRING, string PATTERN, string REPLACEMENT)</code>	Returns the string resulting from replacing all substrings in INITIAL_STRING that match the java regular expression syntax defined in PATTERN with instances of REPLACEMENT.

Name	Description
<code>repeat(string str, int n)</code>	Repeats str n times.
<code>replace(string A, string OLD, string NEW)</code>	Returns the string A with all non-overlapping occurrences of OLD replaced with NEW.
<code>reverse(string A)</code>	Returns the reversed string.
<code>rpad(string str, int len, string pad)</code>	Returns str, right-padded with pad to a length of len.
<code>rtrim(string A)</code>	Returns the string resulting from trimming spaces from the end(right hand side) of A.
<code>sentences(string str, string lang, string locale)</code>	Tokenizes a string of natural language text into words and sentences, where each sentence is broken at the appropriate sentence boundary and returned as an array of words.
<code>space(int n)</code>	Returns a string of n spaces.
<code>split(string str, string pat)</code>	Splits str around pat (pat is a regular expression).
<code>str_to_map(text[, delimiter1, delimiter2])</code>	Splits text into key-value pairs using two delimiters. Delimiter1 separates text into K-V pairs, and Delimiter2 splits each K-V pair. Default delimiters are ‘,’ for delimiter1 and ‘:’ for delimiter2.
<code>substr(string binary A, int start)</code>	
<code>substring_index(string A, string delim, int count)</code>	Returns the substring from string A before count occurrences of the delimiter delim.
<code>translate(string char)</code>	
<code>trim(string A)</code>	Returns the string resulting from trimming spaces from both ends of A.
<code>unbase64(string str)</code>	Converts the argument from a base 64 string to BINARY.

Name	Description
<code>upper(string A) ucase(string A)</code>	Returns the string resulting from converting all characters of A to upper case. For example, <code>upper('fOoBaR')</code> results in 'FOOBAR'.
<code>initcap(string A)</code>	Returns string, with the first letter of each word in uppercase, all other letters in lowercase. Words are delimited by whitespace.
<code>levenshtein(string A, string B)</code>	Returns the Levenshtein distance between two strings.
<code>soundex(string A)</code>	Returns soundex code of the string.
<code>mask(string str[, string upper[, string lower[, string number]]])</code>	Returns a masked version of str.
<code>mask_first_n(string str[, int n])</code>	Returns a masked version of str with the first n values masked. <code>mask_first_n("1234-5678-8765-4321", 4)</code> results in <code>nnnn-5678-8765-4321</code> .
<code>mask_last_n(string str[, int n])</code>	Returns a masked version of str with the last n values masked.
<code>mask_show_first_n(string str[, int n])</code>	Returns a masked version of str, showing the first n characters unmasked.
<code>mask_show_last_n(string str[, int n])</code>	Returns a masked version of str, showing the last n characters unmasked.
<code>mask_hash(string</code>	<code>char</code>
<code>java_method(class, method[, arg1[, arg2..]])</code>	Synonym for <code>reflect</code> .
<code>reflect(class, method[, arg1[, arg2..]])</code>	Calls a Java method by matching the argument signature, using reflection.
<code>hash(a1[, a2...])</code>	Returns a hash value of the arguments.

Name	Description
<code>current_user()</code>	Returns current user name from the configured authenticator manager.
<code>logged_in_user()</code>	Returns current user name from the session state.
<code>current_database()</code>	Returns current database name.
<code>md5(string/binary)</code>	Calculates an MD5 128-bit checksum for the string or binary.
<code>sha1(string/binary)</code> <code>sha(string/binary)</code>	Calculates the SHA-1 digest for string or binary and returns the value as a hex string.
<code>crc32(string/binary)</code>	Computes a cyclic redundancy check value for string or binary argument and returns bigint value.
<code>sha2(string/binary, int)</code>	Calculates the SHA-2 family of hash functions (SHA-224, SHA-256, SHA-384, and SHA-512).
<code>aes_encrypt(input string/binary, key string/binary)</code>	Encrypt input using AES.
<code>aes_decrypt(input binary, key string/binary)</code>	Decrypt input using AES.
<code>version()</code>	Returns the Hive version.
<code>count(expr)</code>	Returns the total number of retrieved rows.
<code>sum(col)</code> , <code>sum(DISTINCT col)</code>	Returns the sum of the elements in the group or the sum of the distinct values of the column in the group.
<code>avg(col)</code> , <code>avg(DISTINCT col)</code>	Returns the average of the elements in the group or the average of the distinct values of the column in the group.
<code>min(col)</code>	Returns the minimum of the column in the group.

Name	Description
<code>max(col)</code>	Returns the maximum value of the column in the group.
<code>variance(col), var_pop(col)</code>	Returns the variance of a numeric column in the group.
<code>var_samp(col)</code>	Returns the unbiased sample variance of a numeric column in the group.
<code>stddev_pop(col)</code>	Returns the standard deviation of a numeric column in the group.
<code>stddev_samp(col)</code>	Returns the unbiased sample standard deviation of a numeric column in the group.
<code>covar_pop(col1, col2)</code>	Returns the population covariance of a pair of numeric columns in the group.
<code>covar_samp(col1, col2)</code>	Returns the sample covariance of a pair of a numeric columns in the group.
<code>corr(col1, col2)</code>	Returns the Pearson coefficient of correlation of a pair of a numeric columns in the group.
<code>percentile(BIGINT col, p)</code>	Returns the exact pth percentile of a column in the group (does not work with floating point types). p must be between 0 and 1.
<code>percentile(BIGINT col, array(p1 [, p2]...))</code>	Returns the exact percentiles p1, p2, ... of a column in the group. pi must be between 0 and 1.
<code>percentile_approx(DOUBLE col, p [, B])</code>	Returns an approximate pth percentile of a numeric column (including floating point types) in the group. The B parameter controls approximation accuracy at the cost of memory. Higher values yield better approximations, and the default is 10,000. When the number of distinct values in col is smaller than B, this gives an exact percentile value.

Name	Description
<code>percentile_approx(DOUBLE col, array(p1 [, p2]...) [, B])</code>	Same as above, but accepts and returns an array of percentile values instead of a single one.
<code>regr_avgx(independent, dependent)</code>	Equivalent to <code>avg(dependent)</code> .
<code>regr_avgy(independent, dependent)</code>	Equivalent to <code>avg(independent)</code> .
<code>regr_count(independent, dependent)</code>	Returns the number of non-null pairs used to fit the linear regression line.
<code>regr_intercept(independent, dependent)</code>	Returns the y-intercept of the linear regression line, i.e. the value of b in the equation $\text{dependent} = a * \text{independent} + b$.
<code>regr_r2(independent, dependent)</code>	Returns the coefficient of determination for the regression.
<code>regr_slope(independent, dependent)</code>	Returns the slope of the linear regression line, i.e. the value of a in the equation $\text{dependent} = a * \text{independent} + b$.
<code>regr_sxx(independent, dependent)</code>	Equivalent to <code>regr_count(independent, dependent) * var_pop(dependent)</code> .
<code>regr_sxy(independent, dependent)</code>	Equivalent to <code>regr_count(independent, dependent) * covar_pop(independent, dependent)</code> .
<code>regr_syy(independent, dependent)</code>	Equivalent to <code>regr_count(independent, dependent) * var_pop(independent)</code> .
<code>histogram_numeric(col, b)</code>	Computes a histogram of a numeric column in the group using b non-uniformly spaced bins. The output is an array of size b of double-valued (x,y) coordinates that represent the bin centers and heights
<code>collect_set(col)</code>	Returns a set of objects with duplicate elements eliminated.
<code>collect_list(col)</code>	Returns a list of objects with duplicates.

Name	Description
<code>ntile(INTEGER x)</code>	Divides an ordered partition into x groups called buckets and assigns a bucket number to each row in the partition. This allows easy calculation of tertiles, quartiles, deciles, percentiles and other common summary statistics.
<code>explode(ARRAY a)</code>	Explodes an array to multiple rows. Returns a row-set with a single column (col), one row for each element from the array.
<code>explode(MAP<Tkey,Tvalue> m)</code>	Explodes a map to multiple rows. Returns a row-set with a two columns (key,value) , one row for each key-value pair from the input map.
<code>posexplode(ARRAY a)</code>	Explodes an array to multiple rows with additional positional column of int type (position of items in the original array, starting with 0). Returns a row-set with two columns (pos,val), one row for each element from the array.
<code>inline(ARRAY<STRUCT<f1:T1,...,fn:Tn>> a)</code>	Explodes an array of structs to multiple rows. Returns a row-set with N columns (N = number of top level elements in the struct), one row per struct from the array.
<code>stack(int r,T1 V1,...,Tn/r Vn)</code>	Breaks up n values V1,...,Vn into r rows. Each row will have n/r columns. r must be constant.
<code>json_tuple(string jsonStr,string k1,...,string kn)</code>	Takes JSON string and a set of n keys, and returns a tuple of n values.
<code>parse_url_tuple(string urlStr,string p1,...,string pn)</code>	Takes URL string and a set of n URL parts, and returns a tuple of n values.

14.5 Modeling

14.5.1 MLlib Functions

The following table exhibits the ML algorithms supported in sparklyr:

14.5.1.1 Classification

Algorithm	Function
Decision Trees	<code>ml_decision_tree_classifier()</code>
Gradient-Boosted Trees	<code>ml_gbt_classifier()</code>
Linear Support Vector Machines	<code>ml_linear_svc()</code>
Logistic Regression	<code>ml_logistic_regression()</code>
Multilayer Perceptron	<code>ml_multilayer_perceptron_classifier()</code>
Naive-Bayes	<code>ml_naive_bayes()</code>
One vs Rest	<code>ml_one_vs_rest()</code>
Random Forests	<code>ml_random_forest_classifier()</code>

14.5.1.2 Regression

Algorithm	Function
Accelerated Failure Time Survival Regression	<code>ml_aft_survival_regression()</code>
Decision Trees	<code>ml_decision_tree_regressor()</code>
Generalized Linear Regression	<code>ml_generalized_linear_regression()</code>
Gradient-Boosted Trees	<code>ml_gbt_regressor()</code>
Isotonic Regression	<code>ml_isotonic_regression()</code>
Linear Regression	<code>ml_linear_regression()</code>

14.5.1.3 Clustering

Algorithm	Function
Bisecting K-Means Clustering	<code>ml_bisecting_kmeans()</code>
Gaussian Mixture Clustering	<code>ml_gaussian_mixture()</code>
K-Means Clustering	<code>ml_kmeans()</code>
Latent Dirichlet Allocation	<code>ml_lda()</code>

14.5.1.4 Recommendation

Algorithm	Function
Alternating Least Squares Factorization	<code>ml_als()</code>

14.5.1.5 Frequent Pattern Mining

Algorithm	Function
FPGrowth	<code>ml_fpgrowth()</code>

14.5.1.6 Feature Transformers

Transformer	Function
Binarizer	<code>ft_binarizer()</code>
Bucketizer	<code>ft_bucketizer()</code>
Chi-Squared Feature Selector	<code>ft_chisq_selector()</code>
Vocabulary from Document Collections	<code>ft_count_vectorizer()</code>
Discrete Cosine Transform	<code>ft_discrete_cosine_transform()</code>
Transformation using dplyr	<code>ft_dplyr_transformer()</code>
Hadamard Product	<code>ft_elementwise_product()</code>
Feature Hasher	<code>ft_feature_hasher()</code>
Term Frequencies using Hashing	<code>export(ft_hashing_tf)</code>
Inverse Document Frequency	<code>ft_idf()</code>
Imputation for Missing Values	<code>export(ft_imputer)</code>
Index to String	<code>ft_index_to_string()</code>
Feature Interaction Transform	<code>ft_interaction()</code>
Rescale to [-1, 1] Range	<code>ft_max_abs_scaler()</code>
Rescale to [min, max] Range	<code>ft_min_max_scaler()</code>
Locality Sensitive Hashing	<code>ft_minhash_lsh()</code>
Converts to n-grams	<code>ft_ngram()</code>
Normalize using the given P-Norm	<code>ft_normalizer()</code>
One-Hot Encoding	<code>ft_one_hot_encoder()</code>
Feature Expansion in Polynomial Space	<code>ft_polynomial_expansion()</code>
Maps to Binned Categorical Features	<code>ft_quantile_discretizer()</code>
SQL Transformation	<code>ft_sql_transformer()</code>
Standardizes Features using Corrected STD	<code>ft_standard_scaler()</code>

Transformer	Function
Filters out Stop Words	<code>ft_stop_words_remover()</code>
Map to Label Indices	<code>ft_string_indexer()</code>
Splits by White Spaces	<code>ft_tokenizer()</code>
Combine Vectors to Row Vector	<code>ft_vector_assembler()</code>
Indexing Categorical Feature	<code>ft_vector_indexer()</code>
Subarray of the Original Feature	<code>ft_vector_slicer()</code>
Transform Word into Code	<code>ft_word2vec()</code>

14.6 Clusters

14.6.1 Google trends for mainframes, cloud computing and kubernetes

Data downloaded from <https://bit.ly/2YnHkNI>.

```
library(dplyr)

read.csv("data/clusters-trends.csv", skip = 2) %>%
  mutate(year = as.Date(paste(Month, "-01", sep = ""))) %>%
  mutate(`On-Premise` = `mainframe...Worldwide`,
         Cloud = `cloud.computing...Worldwide`,
         Kubernetes = `kubernetes...Worldwide`) %>%
  tidyr::gather(`On-Premise`, Cloud, Kubernetes,
               key = "trend", value = "popularity") %>%
  ggplot(aes(x=year, y=popularity, group=trend)) +
  geom_line(aes(linetype = trend, color = trend)) +
  scale_x_date(date_breaks = "2 year", date_labels = "%Y") +
  labs(title = "Cluster Computing Trends",
       subtitle = paste("Search popularity for on-premise (mainframe)",
                        "cloud computing and kubernetes ")) +
  scale_color_grey(start = 0.6, end = 0.2) +
  geom_hline(yintercept = 0, size = 1, colour = "#333333") +
  theme(axis.title.x = element_blank())
```

14.7 Streaming

14.7.1 Stream Generator

The `stream_generate_test()` function creates a local test stream. This function works independently from a Spark connection. The following example will create five files in sub-folder called “source”. The files will be created one second apart from the previous file’s creation.

```
library(sparklyr)

stream_generate_test(iterations = 5, path = "source", interval = 1)
```

After the function completes, all of the files should show up in the “source” folder. Notice that the file size vary. This is so that it simulates what a true stream would do.

```
file.info(file.path("source", list.files("source")))[1]
```

```
##              size
## source/stream_1.csv    44
## source/stream_2.csv   121
## source/stream_3.csv   540
## source/stream_4.csv  2370
## source/stream_5.csv  7236
```

The `stream_generate_test()` by default will create a single numeric variable data frame.

```
readr::read_csv("source/stream_5.csv")
```

```
## # A tibble: 1,489 x 1
##       x
##   <dbl>
## 1    630
## 2    631
## 3    632
## 4    633
## 5    634
## 6    635
## 7    636
## 8    637
## 9    638
## 10   639
## # ... with 1,479 more rows
```

14.7.2 Installing Kafka

This instructions were put together using information from the official Kafka site. Specifically from the Quickstart page, and at the time of writing this book. Newer versions of Kafka undoubtedly will be available when reading this book. The idea is to “timestamp” the versions used in the example in the Streaming chapter.

1. Download Kafka

```
wget http://apache.claz.org/kafka/2.2.0/kafka_2.12-2.2.0.tgz
```

2. Expand the tar file and enter the new folder

```
tar -xzf kafka_2.12-2.2.0.tgz  
cd kafka_2.12-2.2.0
```

3. Start the Zookeeper service that comes with Kafka

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

4. Start the Kafka service

```
bin/kafka-server-start.sh config/server.properties
```

Make sure to always start Zookeeper first, and then Kafka.